Experiment 8: Expression Evaluation using Stack

Problem statement:
WACP to evaluate postfix expression using stack.

i. To study Stack and its operations.
ii. To study the importance of expression evaluation.

Expression evaluation is the process of calculating the value of an arithmetic expression. A stack is the most suitable data structure because it follows the LIFO principle, which matches the order in which operators and operands are processed.
Expression evaluation is commonly performed on:
i. postfix (reverse polish notation)
ii. prefix (polish notation)
Infix expressions are usually converted to postfix or prefix before evaluation.
Eg. Evaluate the expression: 3 5 + 4 *

Symbol	Operation	Stack
3	Push 3	3
5	Push 5	5, 3
+	Pop 5, pop 3, perform 3+5, push 8	8
4	Push 4	4, 8
5	Pop 4, pop 8, perform 4*8, push 32	32

Implementation:

- Platform:

i. 64-bit Open source Linux or its derivatives.

ii. Open Source C Programming tool like gcc/Eclipse Editor.

- Test conditions:

Test Case	Postfix Input	Stack Evaluation Trace	Output
TC01	35+4*	Push 3 & 5, perform 3+5=8, push 4, perform 4*8	32
TC02	532*+	Push 5,3,2, perform 3*2=6, then perform 5+6=11	11
TC03	432*+5-	Push 4,3,2, perform 2*3=6, then 4+6=10, push 5. Perform 10-5=5	5

- Pseudo Code

```
Algorithm Push (opndstk, value):
        if top == MAX - 1
            display "Stack Overflow"
            return
        set top = top + 1
        set opndstk[top] = value
end push

Algorithm Pop(opndstk):
        if top == -1:
            display "Stack Underflow"
            return NULL
        set item = opndstk[top]
        set top = top - 1
        return item
end Pop
```

```
Algorithm EvaluatePostfix (postfixExp):
        Initialilize top = -1
        set i = 0
        while postfixExp[i]!= '\0' (More input items exists):
            set symb = postfixExp[i]
            if symb is an operand (digit/value):
                push(opndstk, numeric_value(symb))
            else if symb is an operator (+, -, *, /, ^):
                opnd2 = Pop(opndstk)
                opnd1 = Pop(opndstk)

                Switch(symb):
                    Case '+':
                        val = opnd1 + opnd2
                    Case '-':
                        val = opnd1 - opnd2
                    Case '*':
                        val = opnd1 * opnd2
                    Case '/':
                        val = opnd1 / opnd2
                    Case '^':
                        val = pow(opnd1, opnd2)
            end switch
            push(opndstk, val)
            set i = i + 1
        set result = Pop(opndstk)
        return result
end EvaluatePostfix
```

- Time complexity:
  Push() / Pop → O(1)
  Evaluate Postfix → O(N)

- Space complexity
  Push() / Pop → O(1)
  EvaluatePostfix → O(N)

- Conclusion:
  Thus, implemented stack operations to evaluate the value of an expression.

FAQs

1. Explain how postfix/prefix expression is evaluated?

Ans. Postfix and prefix are evaluated using an operand stack by scanning the expression tokens sequentially, pushing operands onto the stack, and popping the top two operands whenever an operator is encountered to compute and push back their result.

2. What is ICP and ISP?

Ans. ISP(In-Stack Precedence) represents the precedence level of an operator currently residing inside the stack, while ICP(In-Coming Precedence) represents the precedence level of a newly scanned incoming operator, which together dictate whether to push or operators during expression conversion.

3. Give applications of stack.

Ans. i. Expression evaluation and conversion.
ii. Function call management (call stack) and recursion.
iii. Parentheses matching
iv. Undo/redo operations in text editors
v. Browser navigation history.
vi. Depth-first search (DFS) in graphs.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <math.h>

#define MAX 100

// Stack structure for numerical operands
typedef struct {
    int items[MAX];
    int top;
} Stack;

// Initialize stack
void initStack(Stack *s) {
    s->top = -1;
}

// Check if stack is empty
int isEmpty(Stack *s) {
    return s->top == -1;
}

// Check if stack is full
int isFull(Stack *s) {
    return s->top == MAX - 1;
}

// Push an item onto stack
void push(Stack *s, int val) {
    if (isFull(s)) {
        printf("Stack Overflow!\n");
        return;
```

```
    }
    s->items[++(s->top)] = val;
}

// Pop an item from stack
int pop(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return s->items[(s->top)--];
}

// Evaluate postfix expression
int evaluatePostfix(char *exp) {
    Stack s;
    initStack(&s);

    for (int i = 0; exp[i] != '\0'; i++) {
        char ch = exp[i];

        // Skip spaces if expression uses delimiters
        if (ch == ' ' || ch == '\t') continue;

        // If character is an operand (digit), push to stack
        if (isdigit(ch)) {
            push(&s, ch - '0');
        }
        // If character is an operator, pop two operands and calculate
        else {
            int op2 = pop(&s);
            int op1 = pop(&s);

            switch (ch) {
                case '+': push(&s, op1 + op2); break;
                case '-': push(&s, op1 - op2); break;
```

```
            case '*': push(&s, op1 * op2); break;
            case '/':
                if (op2 == 0) {
                    printf("Error: Division by zero!\n");
                    return -1;
                }
                push(&s, op1 / op2);
                break;
            case '^': push(&s, (int)pow(op1, op2)); break;
            default:
                printf("Invalid operator encountered: %c\n", ch);
                return -1;
        }
    }
  }
  return pop(&s);
}

int main() {
  int choice;
  char postfix[MAX];

  while (1) {
    printf("\n========================================\n");
    printf("  POSTFIX EVALUATION MENU (EXPERIMENT 8) \n");
    printf("========================================\n");
    printf("1. Evaluate Postfix Expression\n");
    printf("2. Exit\n");
    printf("Enter your choice: ");
    if (scanf("%d", &choice) != 1) break;

    switch (choice) {
      case 1:
        printf("Enter Postfix Expression: ");
        scanf("%s", postfix);
        printf("Evaluated Result: %d\n", evaluatePostfix(postfix));
```

```
            break;

        case 2:
            printf("Exiting Postfix Evaluator. Goodbye!\n");
            exit(0);

        default:
            printf("Invalid choice! Please select 1 or 2.\n");
      }
   }

   return 0;
}
```

Input:

Switch option	Resp. Expression
1	35+4*
1	532*+
1	432*+5-
2	Exit

Output:

```
C:\Users\shrey\CLionProjects\untitled\Ass1.exe

========================================
  POSTFIX EVALUATION MENU (EXPERIMENT 8)
========================================
1. Evaluate Postfix Expression
2. Exit
Enter your choice:1

Enter Postfix Expression:35+4*

Evaluated Result: 32

========================================
  POSTFIX EVALUATION MENU (EXPERIMENT 8)
========================================
1. Evaluate Postfix Expression
2. Exit
Enter your choice:1

Enter Postfix Expression:532*+

Evaluated Result: 11

----------------------------------------
```

```

========================================
  POSTFIX EVALUATION MENU (EXPERIMENT 8)
========================================
1. Evaluate Postfix Expression
2. Exit
Enter your choice:1

Enter Postfix Expression:432*+5-

Evaluated Result: 5

========================================
  POSTFIX EVALUATION MENU (EXPERIMENT 8)
========================================
1. Evaluate Postfix Expression
2. Exit
Enter your choice:2

Exiting Postfix Evaluator. Goodbye!

Process finished with exit code 0
```